

TensorFlow Serving for AI Models

High-performance, production-grade model server for deploying machine learning at scale



Why TensorFlow Serving?

The production-ready solution for deploying ML models with:

Performance Optimized

C++ server engineered for **low-latency, high-throughput** inference

Model Management

Native support for **SavedModel** format with built-in versioning

Flexible APIs

gRPC and **REST** endpoints available out of the box

Hardware Flexibility

Works on **CPU and GPU** containers; easy to containerize & scale



SavedModel & Signatures

The deployment package for TensorFlow models:

1 SavedModel Structure

Package models as `SavedModel/` directory containing:

- `assets/` - Additional files needed by the model
- `variables/` - Model weights and variables
- `saved_model.pb` - Model graph definition

2 Signature Definitions

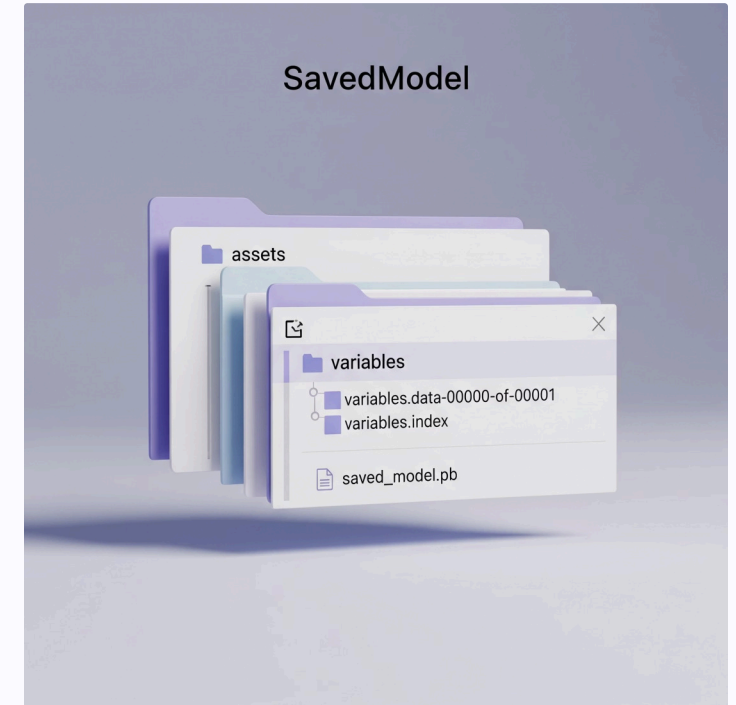
Export a **signature** (e.g., `serving_default`) defining inputs/outputs that clients will use

3 Processing Stability

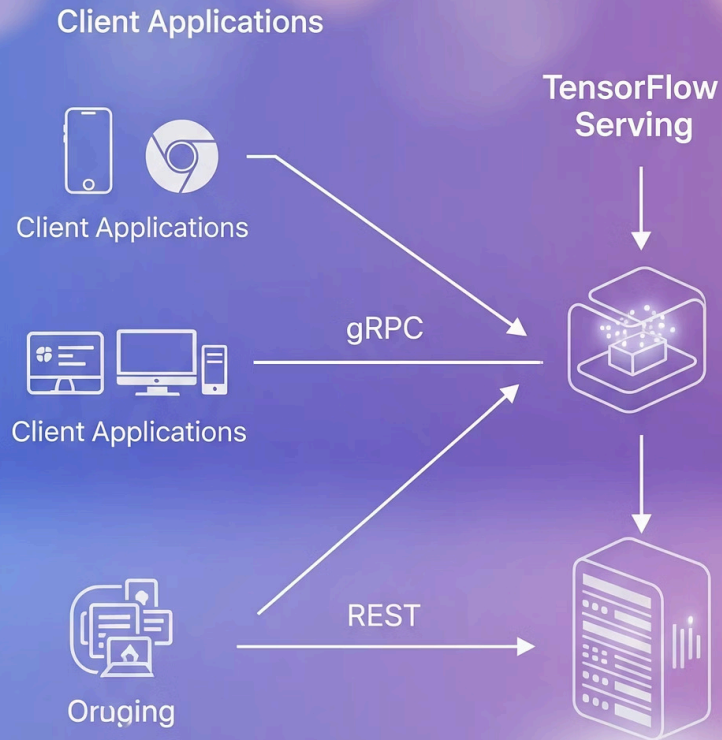
Keep **pre/post-processing** stable; consider separate service or TF ops

4 Version Control

Version each export → enables clean rollbacks and audit trails



Endpoints & Ports



gRPC Endpoint

- Default port: **8500**
- API methods: Predict, Classify, Regress
- Optimized for internal service-to-service
- Higher performance with binary protocol

REST Endpoint

- Default port: **8501**
- API path: `/v1/models/:predict`
- Better for external/edge services
- JSON-based for easier debugging

Important considerations:

- Request shapes **must match** the exported signature definition
- Prefer **gRPC** internally for speed; keep **REST** at the edge if needed
- Both endpoints can be enabled simultaneously on different ports

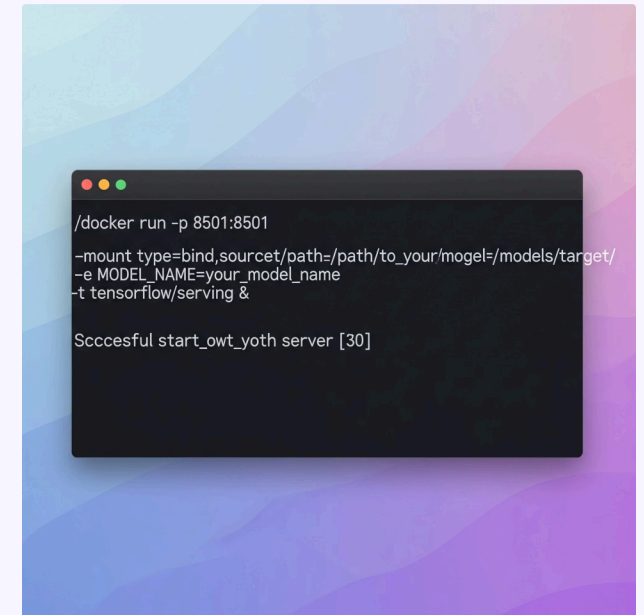
Quickstart (Docker CPU)

Get up and running with TensorFlow Serving in minutes:

```
docker run -p 8501:8501 \  
-v /models/my_model:/models/my_model \  
-e MODEL_BASE_PATH=/models/my_model \  
-e MODEL_NAME=my_model \  
tensorflow/serving:latest
```

Test with REST API:

```
POST /v1/models/my_model:predict  
{  
  "instances": [[...], [...]]  
}
```



Key Components:

- Image: `tensorflow/serving:latest`
- Port mapping: `8501:8501` (REST API)
- Volume mount for model files
- Environment variables for configuration

GPU Serving



Accelerate inference with GPU hardware

For models that benefit from GPU acceleration:

Docker Image

`tensorflow/serving:latest-gpu`

NVIDIA Runtime

Run with `--gpus all` flag in Docker

Environment Variables

`NVIDIA_VISIBLE_DEVICES=all`

`TF_FORCE_GPU_ALLOW_GROWTH=true`

Driver Compatibility

Co-locate with **CUDA/cuDNN** drivers matched to image version

Model Versioning & Policies

Directory Structure

```
/models/  
├── my_model/  
│   ├── 0001/  # Version 1  
│   │   └── saved_model.pb  
│   ├── 0002/  # Version 2  
│   │   └── saved_model.pb  
│   └── 0003/  # Version 3  
│       └── saved_model.pb
```

Version Management Policies



Latest Version

Automatically serves the highest numbered version



Specific Version

Explicitly define which version number to serve



All Versions

Make all versions available through different endpoints

Key Advantage: Hot-swap versions **without downtime** for seamless updates

Multi-Model Config

Serve multiple models from a single TensorFlow Serving instance:

```
# models.config
model_config_list {
  config {
    name: "image_classifier"
    base_path: "/models/image_classifier"
    model_platform: "tensorflow"
  }
  config {
    name: "text_generator"
    base_path: "/models/text_generator"
    model_platform: "tensorflow"
  }
}
```

Start with: `--model_config_file=models.config`



Considerations:

- Monitor **resource budgets** (CPU/GPU memory per model)
- Separate heavy models into distinct deployments for isolation
- Balance model count against available resources
- Consider isolation for critical models

Batching & Throughput

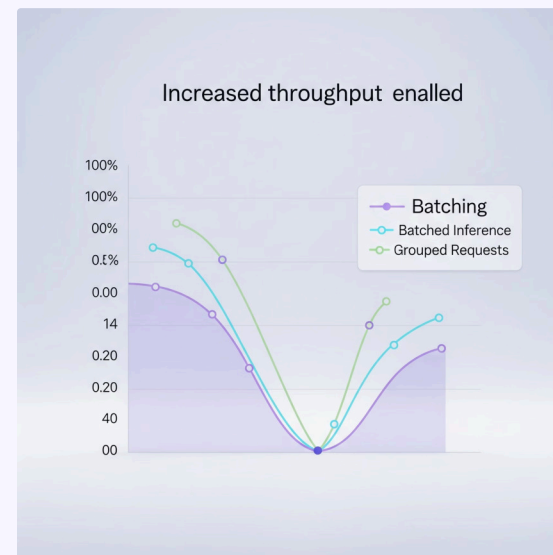
Batching Configuration

Enable batching with: `--enable_batching=true`

Create a batching parameters file:

```
{
  "max_batch_size": 32,
  "batch_timeout_micros": 1000,
  "num_batch_threads": 8,
  "max_enqueued_batches": 100
}
```

Pass with: `--batching_parameters_file=batch_params.txt`



Optimizing Throughput

- Combine with **client-side concurrency** for higher QPS
- Use connection **keep-alive** to avoid handshake overhead
- Always **validate tail latency** (p95/p99) when increasing batch size
- Balance batch size against latency requirements

Performance Tips



Protocol Optimization

Prefer **gRPC** + Protobuf; reuse connections (HTTP/2)

Binary protocol reduces payload size and parsing overhead



Warmup Requests

Add `assets.extra/tf_serving_warmup_requests` file

Eliminates cold-start latency by pre-jitting operations



Precision Optimization

Use **float16/bfloat16** models on GPU when possible

Can double throughput with minimal accuracy impact



Resource Management

Pin CPU threads & use container **CPU/mem limits**

Improves performance predictability in shared environments

Kubernetes Deployment Pattern

Service Architecture

Deployment per model (or per tier), **ClusterIP** service for internal access

Scaling Strategy

HPA on CPU/GPU or custom QPS metrics; **readiness/liveness** probes for health checks

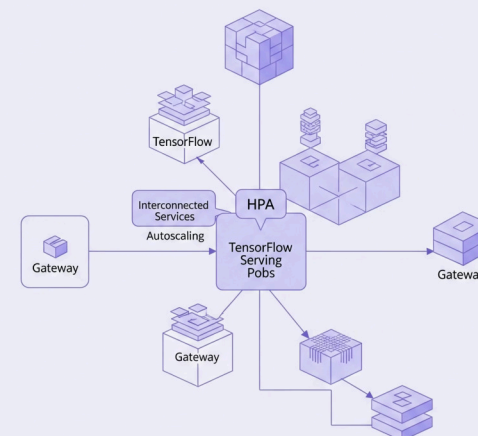
API Gateway

Front with **Envoy/NGINX** for **rate limits, retries, outlier ejection**

Resource Optimization

Use **node/pod affinity** for GPU locality and cache warmth

Kubernetes



Best Practice: Separate critical models into their own deployments for isolation and independent scaling

Observability & Metrics

Key Metrics to Monitor

Performance

- Request rate (QPS)
- Latency (p50/p95/p99)
- Batch size distribution

Resource Utilization

- Memory usage
- CPU/GPU utilization
- Queue size/time

Reliability

- Error rates by type
- Availability percentage
- Model version in use



Observability Stack

- Prometheus metrics endpoint exposed by TF Serving
- Distributed tracing (OpenTelemetry) via gateway/sidecar
- Build **golden dashboards** and SLO alerts before launch
- Correlate model version with performance metrics

Security & Governance

1

Authentication & Authorization

- Terminate TLS at gateway; enforce **mTLS** service-to-service
- AuthZ via JWT/scopes at the edge
- Implement per-tenant quotas and rate limits

2

Model Security

- Sign and verify model artifacts
- Generate SBOM (Software Bill of Materials) for images
- Use least-privilege service accounts

3

Audit & Compliance

- Log **model version** & **request IDs** for audits
- Track model lineage from training to deployment
- Implement input validation boundaries

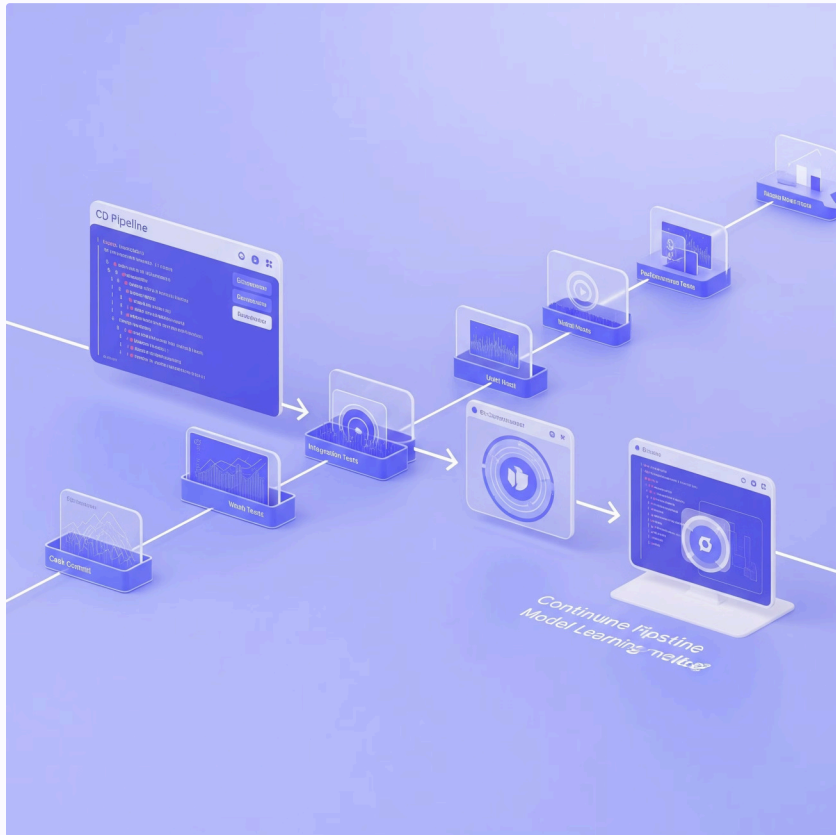
4

Vulnerability Management

- Regular security scans of container images
- Patch cycle for TensorFlow Serving itself
- Network policy enforcement



Testing & Rollouts



Production Readiness Checklist



Contract Testing

Verify **schema** compatibility (inputs/outputs/types) to prevent integration issues



Shadow Testing

Shadow traffic against new versions; compare output distributions



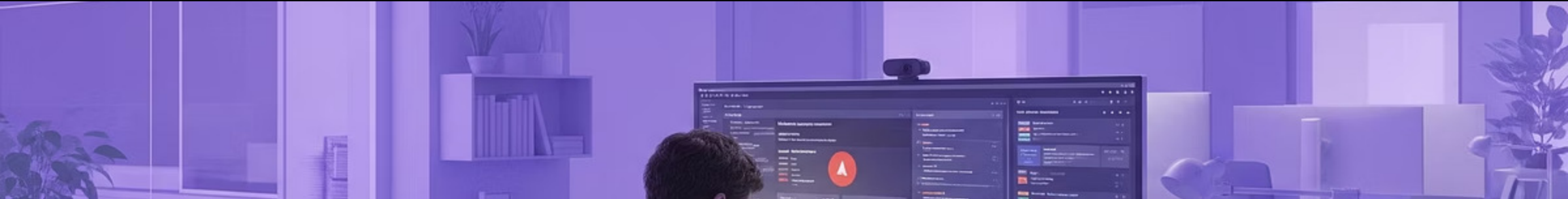
Canary Deployment

Canary 1-5% with fast rollback capabilities; then gradually ramp up



Load Testing

Test with realistic payloads + batching enabled to verify performance at scale



Common Pitfalls



Training/Serving Skew

Different feature logic between training and serving environments

Impact: Inconsistent results in production vs. expectations

Solution: Share preprocessing code between training and serving



Oversized Payloads

Large JSON payloads causing serialization overhead

Impact: Increased latency and reduced throughput

Solution: Switch to **gRPC** or enable compression



Cold Start Issues

No warmup configuration leading to initial request delays

Impact: Latency spikes after deployment or scaling

Solution: Implement proper warmup requests



Resource Contention

Too many models per pod causing resource competition

Impact: Memory thrashing, tail latency inflation

Solution: Right-size deployments and isolate critical models

Key Takeaways

TensorFlow Serving

Production-Ready Platform

Fast, versioned, and production-ready for deploying SavedModel format

Performance Optimization

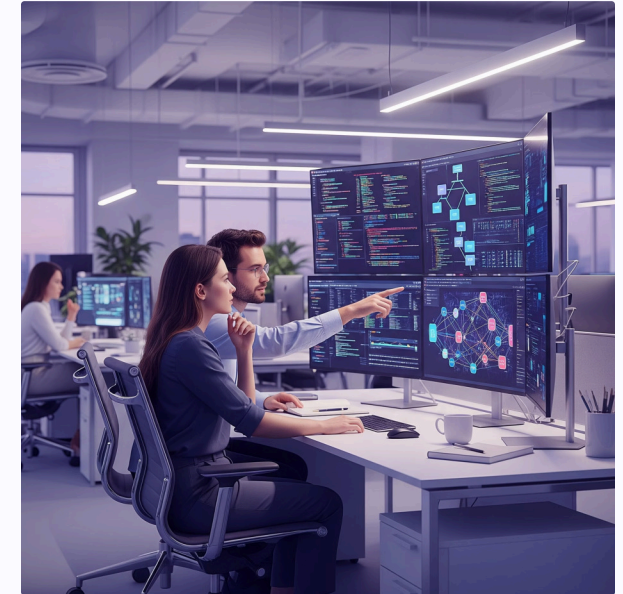
Use **gRPC, batching, and warmup** to maximize throughput and minimize latency

Operational Excellence

Wrap with a **gateway** for security, routing, and policy enforcement

Deployment Methodology

Treat rollout as a product: test, observe, and iterate continuously



Next Steps

- Review your model export process
- Set up a test deployment with TF Serving
- Develop monitoring dashboards
- Create CI/CD pipeline for model updates